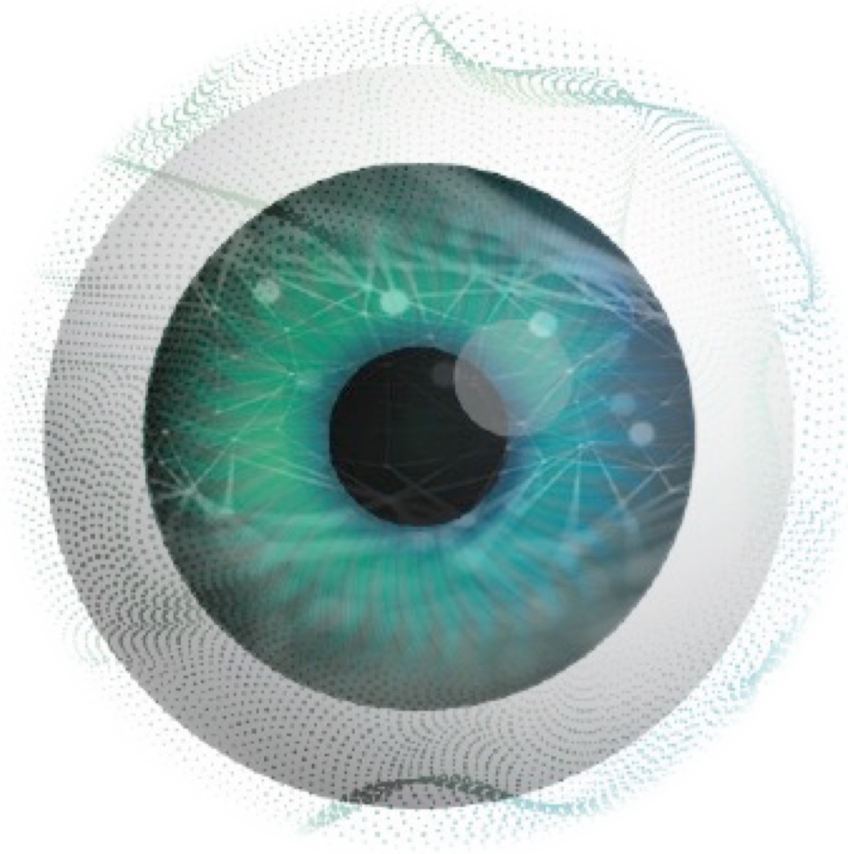
DEEP LEARNING FOR VISUAL RECOGNITION

Lecture 3 – Backpropagation (supplemental)

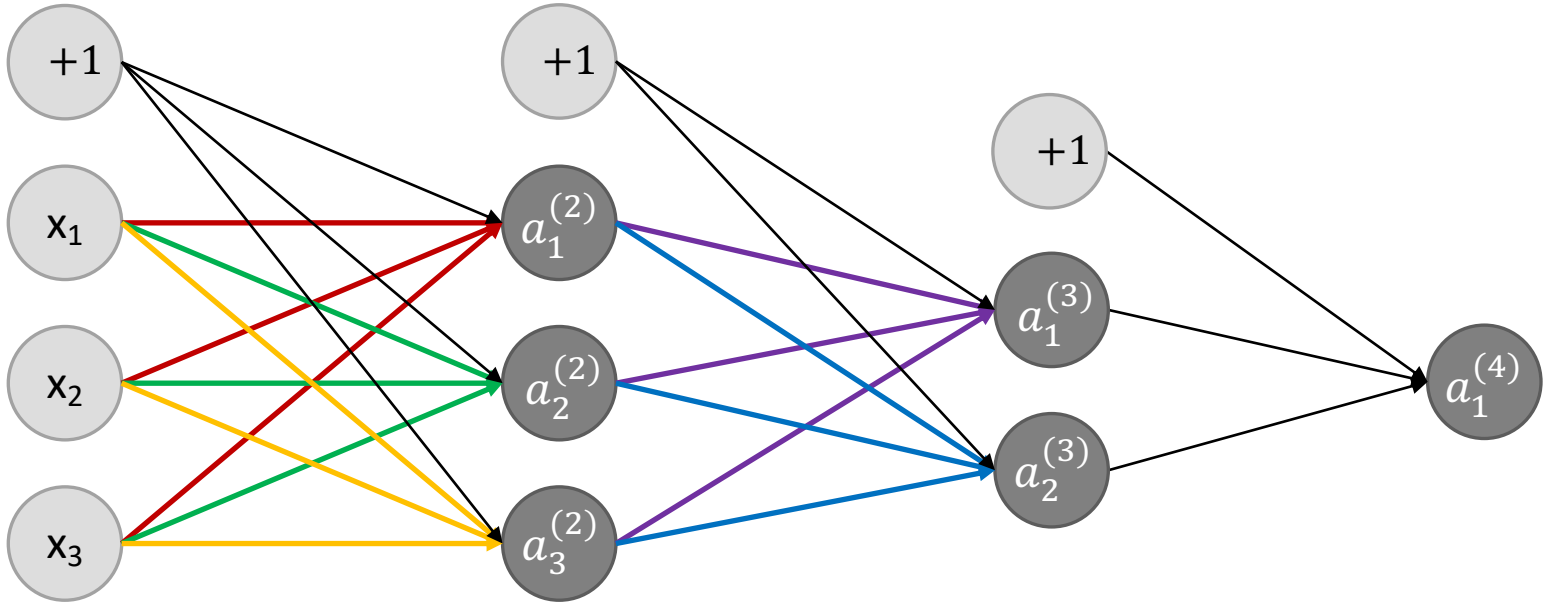


Henrik Pedersen, PhD
External lecturer
Department of Computer Science
Aarhus University
hpe@cs.au.dk

Recap

TRADITIONAL NEURAL NETWORKS

Traditional neural networks



2nd layer features
(based on pixels)

3rd layer features
(based 2nd layer features)

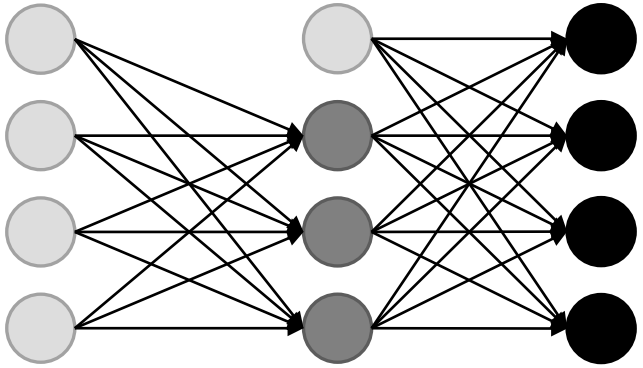
Also commonly referred to as Multilayer Perceptrons or MLPs.

Forward propagation:

$$a^{(1)} = x$$
$$z^{(j+1)} = W^{(j)} a^{(j)} + b^{(j)}$$
$$a^{(j+1)} = \sigma\left(z^{(j+1)}\right)$$

Training data for multiclass classification

- The training set is $\left\{y^{(k)}, x^{(k)}\right\}_{k=1}^{n}$
- where $y^{(k)}$ is a one-hot vector, like $\begin{bmatrix}1\\0\\0\\0\end{bmatrix}$ $\begin{bmatrix}0\\1\\0\\0\end{bmatrix}$ $\begin{bmatrix}0\\0\\1\\0\end{bmatrix}$ $\begin{bmatrix}0\\0\\0\\1\end{bmatrix}$
- **Goal of training:** Learn parameters such that $h_{W,b}\left(x^{(k)}\right) = y^{(k)}$



$$h_{W,b}(x) = \begin{bmatrix} P(y = Pedestrain|x) \\ P(y = Car|x) \\ P(y = Motorcycle|x) \\ P(y = Truck|x) \end{bmatrix}$$

Examples of loss functions

- Cross entropy loss:

$$J(W,b) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)} \log\left(h_{W,b}\left(x^{(i)}\right)_k\right)$$

- Extended cross entropy loss:

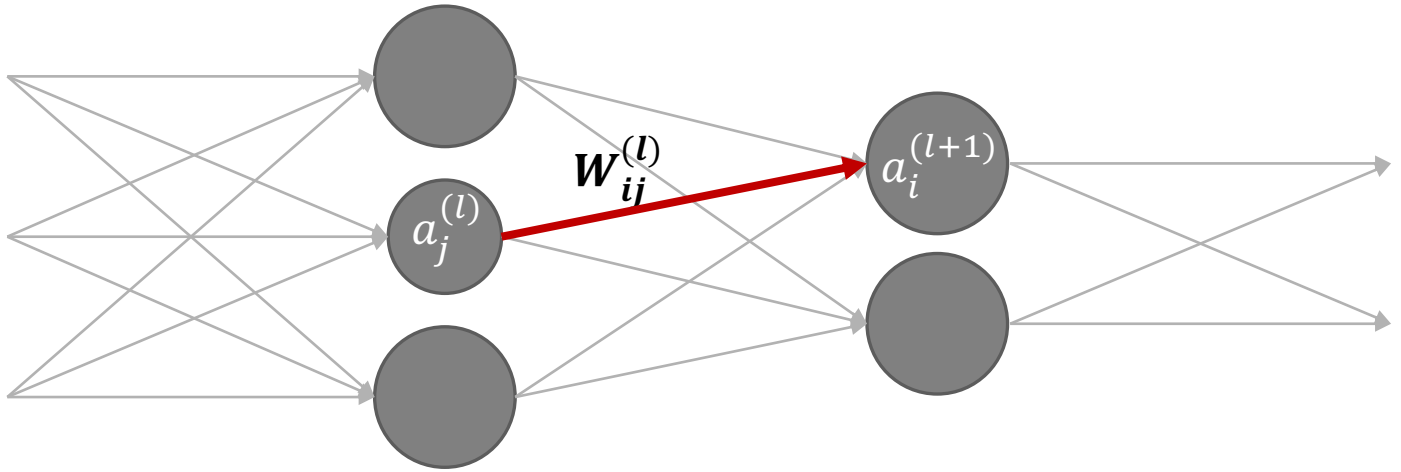
$$J(W,b) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)} \log\left(h_{W,b}\left(x^{(i)}\right)_k\right) + \left(1 - y_k^{(i)}\right) \log\left(1 - h_{W,b}\left(x^{(i)}\right)_k\right)$$

- Quadratic loss:

$$J(W,b) = \frac{1}{2}\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K}\left(h_{W,b}\left(x^{(i)}\right)_k - y_k^{(i)}\right)^2$$

Brute-force gradient approximation

- To minimize the loss using gradient descent we need to be able to compute $\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}}$ and $\frac{\partial J(W,b)}{\partial b_i^{(l)}}$
- Brute-force gradient approximation: $\frac{\partial J(W)}{\partial W_{ij}^{(l)}} \approx \frac{J\left(W + \varepsilon e_{ij}^{(l)}\right) - J(W)}{\varepsilon}$
- **Problem:** Extremely slow in practise
- **Solution:** Backpropagation = one forward pass + one backward pass to calculate all partial derivatives.



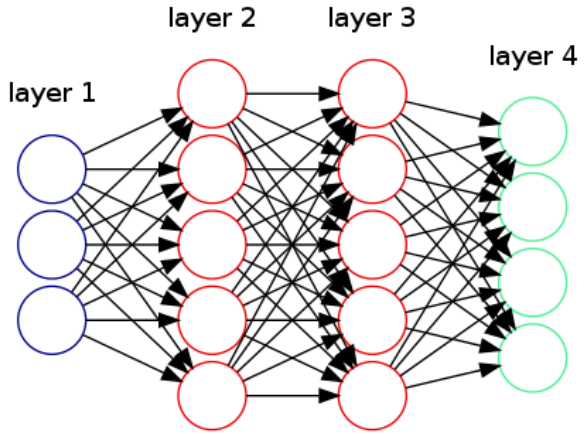
Loss:
$J(W)$

Backpropagation

MANUAL DIFFERENTIATION

Let's consider this specific neural network

- L: Total number of layers
- s_l: No. of units (not counting bias unit) in layer l.
- $(W, b) = \left(W_{ij}^{(l)}, b_i^{(l)}\right)_{l=1}^{L}$: Model parameters, where we write $W_{ij}^{(l)}$ to denote the weight associated with the connection between unit j in layer l, and unit i in layer $l+1$ (note the order of the indices). Also, $b_i^{(l)}$ is the bias associated with unit i in layer $l+1$. $W^{(l)}$ has size $s_{l+1} \times s_l$ and $b^{(l)}$ has length s_{l+1}.



Example:

$L = 4, s_1 = 3, s_2 = 5, s_3 = 5, s_4 = 4$

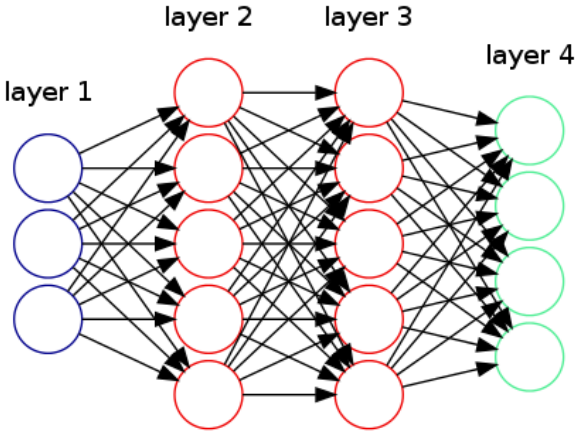
$W^{(1)} \in \mathbb{R}^{5\times3}, W^{(2)} \in \mathbb{R}^{5\times5}, W^{(3)} \in \mathbb{R}^{4\times5}$

$b^{(1)} \in \mathbb{R}^5, b^{(2)} \in \mathbb{R}^5, b^{(3)} \in \mathbb{R}^4$

Forward propagation

$x^{(k)}$ is the *k*'th training example

$$
\begin{aligned}
a^{(1)} &= x^{(k)} \\
z^{(2)} &= W^{(1)} a^{(1)} + b^{(1)} \\
a^{(2)} &= \sigma\left(z^{(2)}\right) \\
z^{(3)} &= W^{(2)} a^{(2)} + b^{(2)} \\
a^{(3)} &= \sigma\left(z^{(3)}\right) \\
z^{(4)} &= W^{(3)} a^{(3)} + b^{(3)} \\
a^{(4)} &= \sigma\left(z^{(4)}\right) \\
&= h_{W,b}\left(x^{(k)}\right)
\end{aligned}
$$



$$
\begin{aligned}
a^{(1)} &= x \\
z^{(l+1)} &= W^{(l)} a^{(l)} + b^{(l)} \\
a^{(l+1)} &= \sigma\left(z^{(l+1)}\right)
\end{aligned}
$$

$y^{(k)}$ is the target/label of $x^{(k)}$

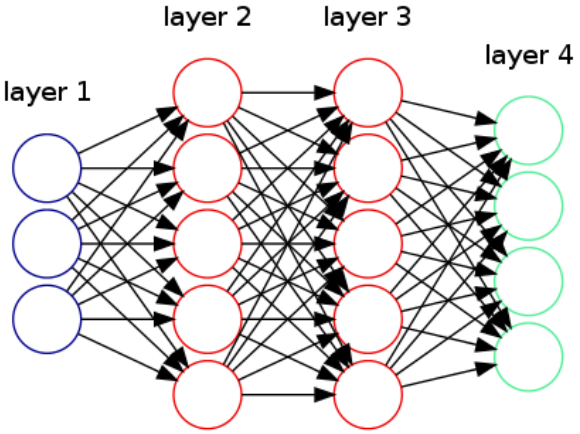
Goal of training:

Learn parameters such that $h_{W,b}\left(x^{(k)}\right) = y^{(k)}$

Backpropagation

- **Intuition:** $\delta_j^{(l)}$ = "error" of unit j in layer l.
- $\delta^{(l)}$ is a vector

$$\delta^{(4)} = \left(a^{(4)} - y^{(k)}\right) \odot \sigma'\left(z^{(4)}\right)$$
$$\delta^{(3)} = \left(W^{(3)}\right)^T \delta^{(4)} \odot \sigma'\left(z^{(3)}\right)$$
$$\delta^{(2)} = \left(W^{(2)}\right)^T \delta^{(3)} \odot \sigma'\left(z^{(2)}\right)$$



- There is no $\delta^{(1)}$ term, because layer 1 corresponds to the input (x).
- $\odot$ denotes element-wise matrix or vector multiplication
- And $\sigma'\left(z^{(j)}\right) = \sigma\left(z^{(j)}\right) \odot \left(1 - \sigma\left(z^{(j)}\right)\right) = a^{(j)} \odot \left(1 - a^{(j)}\right)$ is the derivate of the sigmoid function (σ).

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = a_j^{(l)} \delta_i^{(l+1)}$$

$$\frac{\partial J(W,b)}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$$

Backpropagation algorithm

- The training set is $\{y^{(k)}, x^{(k)}\}_{k=1}^{n}$
- Set $\Delta W_{ij}^{(l)} = 0$ (for all i, j, l) and $\Delta b_i^{(l)} = 0$ (for all i, l)
- For $k = 1$ to n:
  - Set $a^{(1)} = x^{(k)}$
  - Perform forward propagation to compute $a^{(l)}$ for all $l = 2, \ldots, L$
  - Using $y^{(k)}$, compute $\delta^{(L)} = \left(a^{(L)} - y^{(k)}\right) \odot \sigma'\left(z^{(L)}\right)$
  - Compute $\delta^{(L-1)}, \delta^{(L-2)}, \ldots, \delta^{(2)}$
  - $\Delta W_{ij}^{(l)} = \Delta W_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ for all i, j in layers $l = 1, \ldots, L-1$
  - $\Delta b_i^{(l)} = \Delta b_i^{(l)} + \delta_i^{(l+1)}$ for all i in layers $l = 1, \ldots, L-1$
- Update parameters (gradient descent)

Note that we use *k* to index over training examples. We cannot use *i* here, because *i* refers to a unit in the network.

Backpropagation algorithm

- The training set is $\{y^{(k)}, x^{(k)}\}_{k=1}^{n}$
- Set $\Delta W_{ij}^{(l)} = 0$ (for all i, j, l) and $\Delta b_i^{(l)} = 0$ (for all i, l)
- For $k = 1$ to n:
  - Set $a^{(1)} = x^{(k)}$
  - Perform forward propagation to compute $a^{(l)}$ for all $l = 2, \ldots, L$
  - Using $y^{(k)}$, compute $\delta^{(L)} = \left(a^{(L)} - y^{(k)}\right) \odot \sigma'\left(z^{(L)}\right)$
  - Compute $\delta^{(L-1)}, \delta^{(L-2)}, \ldots, \delta^{(2)}$
  - $\Delta W_{ij}^{(l)} = \Delta W_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ for all i, j in layers $l = 1, \ldots, L-1$
  - $\Delta b_i^{(l)} = \Delta b_i^{(l)} + \delta_i^{(l+1)}$ for all i in layers $l = 1, \ldots, L-1$
- Update parameters (gradient descent)

Calculating the gradients

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = \frac{1}{n} \Delta W_{ij}^{(l)}$$

$$\frac{\partial J(W,b)}{\partial b_i^{(l)}} = \frac{1}{n} \Delta b_i^{(l)}$$

Backpropagation algorithm

- The training set is $\{y^{(k)}, x^{(k)}\}_{k=1}^{n}$
- Set $\Delta W_{ij}^{(l)} = 0$ (for all i, j, l) and $\Delta b_i^{(l)} = 0$ (for all i, l)
- For $k = 1$ to n:
  - Set $a^{(1)} = x^{(k)}$
  - Perform forward propagation to compute $a^{(l)}$ for all $l = 2, \ldots, L$
  - Using $y^{(k)}$, compute $\delta^{(L)} = \left(a^{(L)} - y^{(k)}\right) \odot \sigma'\left(z^{(L)}\right)$
  - Compute $\delta^{(L-1)}, \delta^{(L-2)}, \ldots, \delta^{(2)}$
  - $\Delta W_{ij}^{(l)} = \Delta W_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ for all i, j in layers $l = 1, \ldots, L-1$
  - $\Delta b_i^{(l)} = \Delta b_i^{(l)} + \delta_i^{(l+1)}$ for all i in layers $l = 1, \ldots, L-1$
- Update parameters (gradient descent)

Calculating the gradients

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = \frac{1}{n} \Delta W_{ij}^{(l)}$$

$$\frac{\partial J(W,b)}{\partial b_i^{(l)}} = \frac{1}{n} \Delta b_i^{(l)}$$

Parameter update (gradient descent)

$$W_{ij}^{(l)} = W_{ij}^{(l)} - \alpha \frac{\partial J(W,b)}{\partial W_{ij}^{(l)}}$$

$$b_i^{(l)} = b_i^{(l)} - \alpha \frac{\partial J(W,b)}{\partial b_i^{(l)}}$$

Backpropagation algorithm

- The training set is $\{y^{(k)}, x^{(k)}\}_{k=1}^{n}$
- Set $\Delta W_{ij}^{(l)} = 0$ (for all i, j, l) and $\Delta b_i^{(l)} = 0$ (for all i, l)
- For $k = 1$ to n:
  - Set $a^{(1)} = x^{(k)}$
  - Perform forward propagation to compute $a^{(l)}$ for all $l = 2, \ldots, L$
  - Using $y^{(k)}$, compute $\delta^{(L)} = \left(a^{(L)} - y^{(k)}\right) \odot \sigma'\left(z^{(L)}\right)$
  - Compute $\delta^{(L-1)}, \delta^{(L-2)}, \ldots, \delta^{(2)}$
  - $\Delta W_{ij}^{(l)} = \Delta W_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ for all i, j in layers $l = 1, \ldots, L-1$
  - $\Delta b_i^{(l)} = \Delta b_i^{(l)} + \delta_i^{(l+1)}$ for all i in layers $l = 1, \ldots, L-1$
- Update parameters (gradient descent)

Vectorized computations

$$\Delta W^{(l)} = \Delta W^{(l)} + \delta^{(l+1)} \left(a^{(l)}\right)^T$$

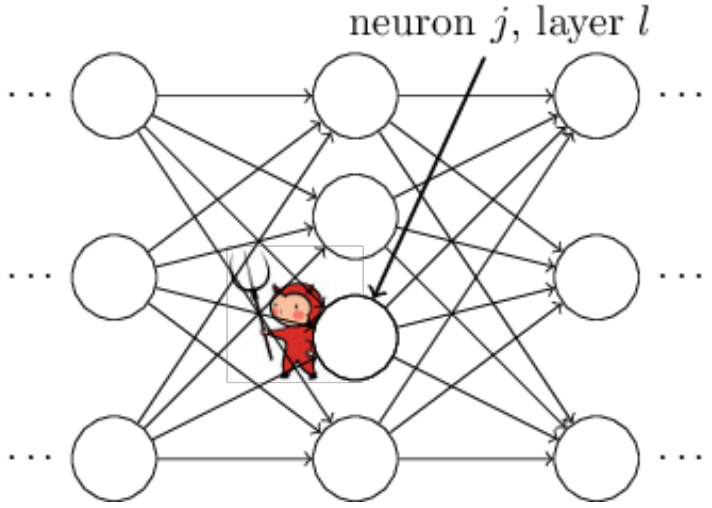
$$\Delta b^{(l)} = \Delta b^{(l)} + \delta^{(l+1)}$$

Backpropagation intuition

- To understand where the "error" $\delta_j^{(l)}$ comes from, imagine there is a demon in our neural net:
- The demon sits at the j'th neuron in layer l. As the input to the neuron comes in, the demon messes with the neuron's operation. It adds a little change $\Delta z_j^{(l)}$ to the neuron's weighted input, so that instead of outputting $\sigma\left(z_j^{(l)}\right)$, the neuron instead outputs $\sigma\left(z_j^{(l)} + \Delta z_j^{(l)}\right)$.
- This change propagates through later layers in the network, finally causing the overall loss to change by an amount

$$\frac{\partial J(W,b)}{\partial z_j^{(l)}} \Delta z_j^{(l)}$$

- Defining $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$, we can use it to figure out how to set $\Delta z_j^{(l)}$, such that the loss is minimized.



http://neuralnetworksanddeeplearning.com/chap2.html

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- We will assume that we have just one training example (x,y) and prove backpropagation for the simple quadratic loss

$$J(W,b) = \frac{1}{2}\sum_{k=1}^{s_L}\left(h_{W,b}(x)_k - y_k\right)^2 = \frac{1}{2}\sum_{k=1}^{s_L}\left(a_k^{(L)} - y_k\right)^2$$

- We will be needing the derivative w.r.t. $a_j^{(L)}$:

$$\frac{\partial J(W,b)}{\partial a_j^{(L)}} = a_j^{(L)} - y_j$$

Proof: Last layer

- Let's define c

- We will start

Chain rule:

If $z = f(y)$ and $y = g(x)$, then

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

$y = (x - c)$: $\frac{dy}{dx} = 1$ and $z = \frac{1}{2}y^2$: $\frac{dz}{dy} = y = x - c$

- We will assume that we have just one training example (x,y) and prove backpropagation for the simple quadratic loss

$$J(W,b) = \frac{1}{2}\sum_{k=1}^{s_L}\left(h_{W,b}(x)_k - y_k\right)^2 = \frac{1}{2}\sum_{k=1}^{s_L}\left(a_k^{(L)} - y_k\right)^2$$

- We will be needing the derivative w.r.t. $a_j^{(L)}$:

$$\frac{\partial J(W,b)}{\partial a_j^{(L)}} = a_j^{(L)} - y_j$$

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- **Intuition:**
  - The first term on the right is actually the partial derivative of the loss w.r.t. to the j'th output activation:

$$\partial J(W,b)/\partial a_j^{(L)} = a_j^{(L)} - y_j$$

  - It measures how fast the loss is changing as a function of the j'th output activation. If, for example, J doesn't depend much on a particular output neuron, j, then $\delta_j^{(l)}$ will be small, which is what we'd expect.
  - The second term on the right, $\sigma'\left(z_j^{(L)}\right)$, measures how fast the activation function σ is changing at $z_j^{(L)}$.

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

Chain rule:
If $z = f(y)$ and $y = g(x)$, then

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

- **Proof:**

$$\delta_j^{(L)} = \frac{\partial J(W,b)}{\partial z_j^{(L)}} = \sum_{k=1}^{s_L} \frac{\partial J(W,b)}{\partial a_k^{(L)}}\frac{\partial a_k^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}}\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}}\sigma'\left(z_j^{(L)}\right) = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- Where we have used the chain rule of differentiation and that

$$a_j^{(L)} = \sigma\left(z_j^{(L)}\right) \rightarrow \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \sigma'\left(z_j^{(L)}\right) \qquad \frac{\partial J(W,b)}{\partial a_j^{(L)}} = a_j^{(L)} - y_j$$

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- **Proof:**

$$\delta_j^{(L)} = \frac{\partial J(W,b)}{\partial z_j^{(L)}} = \sum_{k=1}^{s_L} \frac{\partial J(W,b)}{\partial a_k^{(L)}} \frac{\partial a_k^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}} \sigma'\left(z_j^{(L)}\right) = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- Where we have used the chain rule of differentiation and that

$$a_j^{(L)} = \sigma\left(z_j^{(L)}\right) \rightarrow \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \sigma'\left(z_j^{(L)}\right) \qquad \frac{\partial J(W,b)}{\partial a_j^{(L)}} = a_j^{(L)} - y_j$$

Q: All terms in the summation are zero, except when k = j. So why did we put the summation there in the first place?
A: During forward propagation, the calculation of $z_j^{(L)}$ appears earlier than $a_k^{(L)}$. Pretending that we don't know the exact NN architecture, we cannot exclude the possibility that $z_j^{(L)}$ affects $a_k^{(L)}$ for all k. This would imply that $\frac{\partial J(W,b)}{\partial z_j^{(L)}}$ would be distributed over all terms involving $a_k^{(L)}$. We are simply summing over all of these terms.

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- **Proof:**

$$\delta_j^{(L)} = \frac{\partial J(W,b)}{\partial z_j^{(L)}} = \sum_{k=1}^{s_L} \frac{\partial J(W,b)}{\partial a_k^{(L)}} \frac{\partial a_k^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \frac{\partial J(W,b)}{\partial a_j^{(L)}} \sigma'\left(z_j^{(L)}\right) = \left(a_j^{(L)} - y_j\right)\sigma'\left(z_j^{(L)}\right)$$

- Where we have used the chain rule of differentiation and that

$$a_j^{(L)} = \sigma\left(z_j^{(L)}\right) \rightarrow \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \sigma'\left(z_j^{(L)}\right) \qquad \frac{\partial J(W,b)}{\partial a_j^{(L)}} = a_j^{(L)} - y_j$$

Proof: Last layer

- Let's define $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l.
- We will start from the last layer of the network and show that

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right) \sigma'\left(z_j^{(L)}\right)$$

- **Vectorized form:**

$$\delta^{(L)} = \left(a^{(L)} - y\right) \odot \sigma'\left(z^{(L)}\right)$$

Proof: Intermediate layers

- Now, lets show that for the other layers

$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right)$$

- **Intuition:**
  - This equation appears complicated, but each element has a nice interpretation.
  - Suppose we know the error $\delta^{(l+1)}$ at the $l+1$'th layer (recall that we are doing backpropagation). When we apply the transpose weight matrix, $\left(W^{(l)}\right)^T$, we can think intuitively of this as **moving the error backward through the network**, giving us some sort of measure of the error at the output of the l'th layer.
  - We then take the element-wise product $\odot \sigma'\left(z^{(l)}\right)$. This moves the error backward through the activation function in layer l, giving us the error $\delta^{(l)}$ in the weighted input to layer l.

Proof: Intermediate layers

- Now, lets show that for the other layers

$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right)$$

- **Proof:**

$$\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \frac{\partial J(W,b)}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

- Where we have used the chain rule and the definition of the next layer's error: $\delta_k^{(l+1)} = \frac{\partial J(W,b)}{\partial z_k^{(l+1)}}$

Proof: Intermediate layers

- Now, lets show that for the other layers

$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right)$$

- **Proof:**

$$\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \frac{\partial J(W,b)}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

- Furthermore, we have

$$z_k^{(l+1)} = \sum_i W_{ki}^{(l)} a_i^{(l)} + b_k^{(l)} = \sum_i W_{ki}^{(l)} \sigma\left(z_i^{(l)}\right) + b_k^{(l)} \quad \rightarrow \quad \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l)} \sigma'\left(z_j^{(l)}\right)$$

Proof: Intermediate layers

- Now, lets show that for the other layers

$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right)$$

- **Proof:**

$$\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \frac{\partial J(W,b)}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l)} \sigma'\left(z_j^{(l)}\right)$$

$$\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l)} \sigma'\left(z_j^{(l)}\right)$$

Proof: Intermediate layers

- Now, lets show that for the other layers

$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right)$$

- **Proof:**

$$\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \frac{\partial J(W,b)}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{s_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l)} \sigma'\left(z_j^{(l)}\right)$$

- This is the component-wise error.
- Rearranging terms and vectorizing gives the desired result.

Proof: Weight update

- Now, let's prove that

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = a_j^{(l)} \delta_i^{(l+1)}$$

- Proof:

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = \frac{\partial J(W,b)}{\partial z_i^{(l+1)}} \frac{\partial z_i^{(l+1)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l+1)} \frac{\partial z_i^{(l+1)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l+1)} a_j^{(l)}$$

- Where we have used the chain rule and the fact that

$$z_i^{(l+1)} = \sum_k W_{ik}^{(l)} a_k^{(l)} + b_i^{(l)} \quad \rightarrow \quad \frac{\partial z_i^{(l+1)}}{\partial W_{ij}^{(l)}} = a_j^{(l)}$$

Proof: Bias update

- Now, let's prove that

$$\frac{\partial J(W,b)}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$$

- Proof:

$$\frac{\partial J(W,b)}{\partial b_i^{(l)}} = \frac{\partial J(W,b)}{\partial z_i^{(l+1)}} \frac{\partial z_i^{(l+1)}}{\partial b_i^{(l)}} = \delta_i^{(l+1)} \frac{\partial z_i^{(l+1)}}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$$

- Where we have used the chain rule and the fact that

$$z_i^{(l+1)} = \sum_k W_{ik}^{(l)} a_k^{(l)} + b_i^{(l)} \quad \rightarrow \quad \frac{\partial z_i^{(l+1)}}{\partial b_i^{(l)}} = 1$$

Backpropagation summary

- Defining $\delta_j^{(l)} = \frac{\partial J(W,b)}{\partial z_j^{(l)}}$ = "error" of unit j in layer l, we have just shown that

$$\delta^{(L)} = \left(a^{(L)} - y\right) \odot \sigma'\left(z^{(L)}\right) = \left(h_{W,b}(x) - y\right) \odot \sigma'\left(z^{(L)}\right)$$
$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right) \text{ for } l = L-1, L-2, \ldots, 2$$

- where $\sigma'\left(z^{(l)}\right) = \sigma\left(z^{(l)}\right) \odot \left(1 - \sigma\left(z^{(l)}\right)\right) = a^{(l)} \odot \left(1 - a^{(l)}\right)$
- We have also shown that the partial derivatives can be computes as

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = a_j^{(l)} \delta_i^{(l+1)} \text{ and } \frac{\partial J(W,b)}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$$

Again, why do we even need backprop?

- Suppose we want to compute the partial derivative of the loss function with respect to some weight w_j. We could approximate the gradient like this:

$$\frac{\partial J(w)}{\partial w_j} \approx \frac{J\left(w + \varepsilon e_j\right) - J(w)}{\varepsilon}$$

- where e_j is a unit vector pointing in the direction of w_j, and ε is a very small number.
- Unfortunately, when you implement the code it turns out to be **extremely slow**. To understand why, imagine we have a million weights in our network. Then for each distinct weight w_j we need to compute $J\left(w + \varepsilon e_j\right)$ in order to compute $\partial J(w)/\partial w_j$. That means that **to compute the gradient we need to compute the cost function a million different times**, requiring a million forward passes through the network (per training example).
- What's clever about backpropagation is that it enables us to simultaneously compute all the partial derivatives $\partial J(w)/\partial w_j$ using just one forward pass through the network, followed by one backward pass through the network.

Computational Graphs

AUTOMATIC DIFFERENTIATION

Backpropagation revisited

- For traditional neural networks, we manually derived that if we define

$$\delta^{(L)} = \left(a^{(L)} - y\right) \odot \sigma'\left(z^{(L)}\right) = \left(h_{W,b}(x) - y\right) \odot \sigma'\left(z^{(L)}\right)$$
$$\delta^{(l)} = \left(W^{(l)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right) \text{ for } l = L-1, L-2, \ldots, 2$$

- then

$$\frac{\partial J(W,b)}{\partial W_{ij}^{(l)}} = a_j^{(l)} \delta_i^{(l+1)} \text{ and } \frac{\partial J(W,b)}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$$

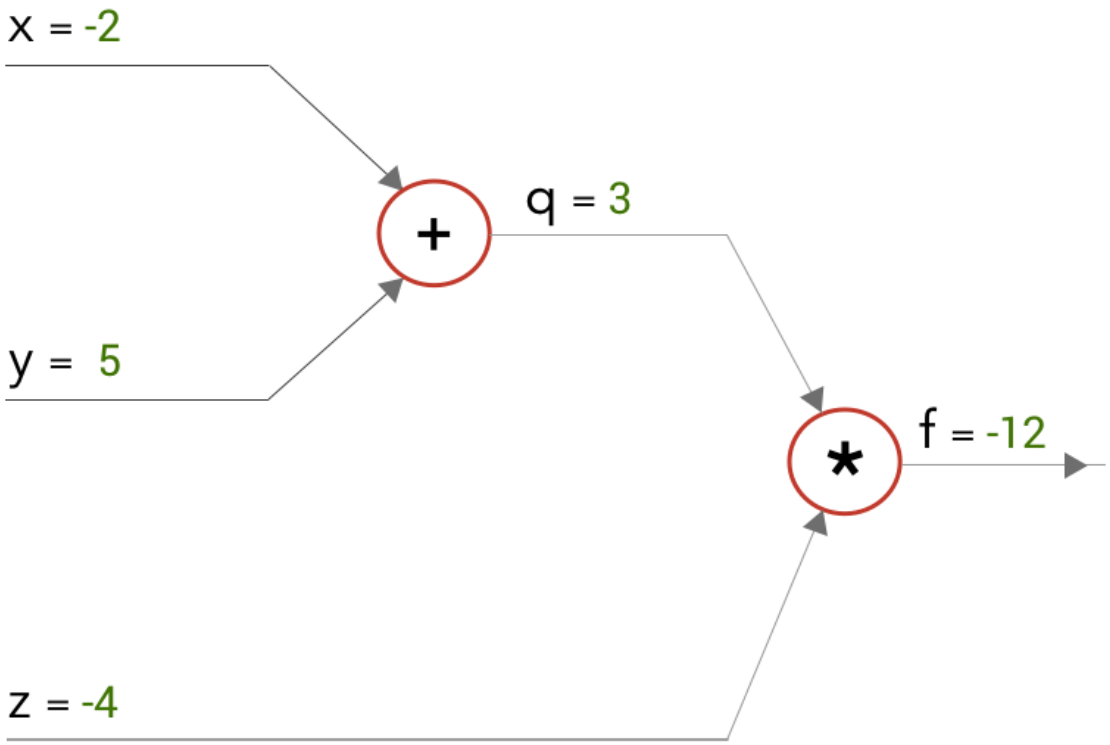
- **Problem:** If we change the network architecture (e.g., by adding convolutional layers) or the loss function, we have to derive everything again.
- **Solution:** Computational graphs and auto-differentiation

Computational graphs

- All modern deep learning frameworks use computational graphs.
- Basically, they enable automatic calculation of all partial derivatives.
- All you have to do is specify the underlying **computational graph** of your network.
- Really nice intro here: https://medium.com/@pavisj/convolutions-and-backpropagations-46026a8f5d2c
- Other resources
  - https://blog.paperspace.com/pytorch-101-understanding-graphs-and-automatic-differentiation/
  - https://towardsdatascience.com/pytorch-autograd-understanding-the-heart-of-pytorchs-magic-2686cd94ec95
  - https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
  - https://pytorch.org/tutorials/beginner/examples_autograd/two_layer_net_custom_function.html

Example

- Consider the equation $f(x, y, z) = (x + y)z$
- Our goal is to **calculate the gradients**: ∂f/∂x, ∂f/∂y and ∂f/∂z.
- To make it simpler, let us split it into two equations.
  - $q = x + y$
  - $f = q * z$
- Now, let us draw a computational graph for it with values x = -2, y = 5, z = 4.
- **Forward pass** results in f = - 12.
- Now, let's do the **backward pass** to calculate the gradients.



Example

- Working from right to left, at the multiply gate we can differentiate f to get the gradients at q and z — ∂f/∂q and ∂f/∂z.
- And at the add gate, we can differentiate q to get the gradients at x and y — ∂q/∂x and ∂q/∂y.
- How do we compute ∂f/∂x and ∂f/∂y?

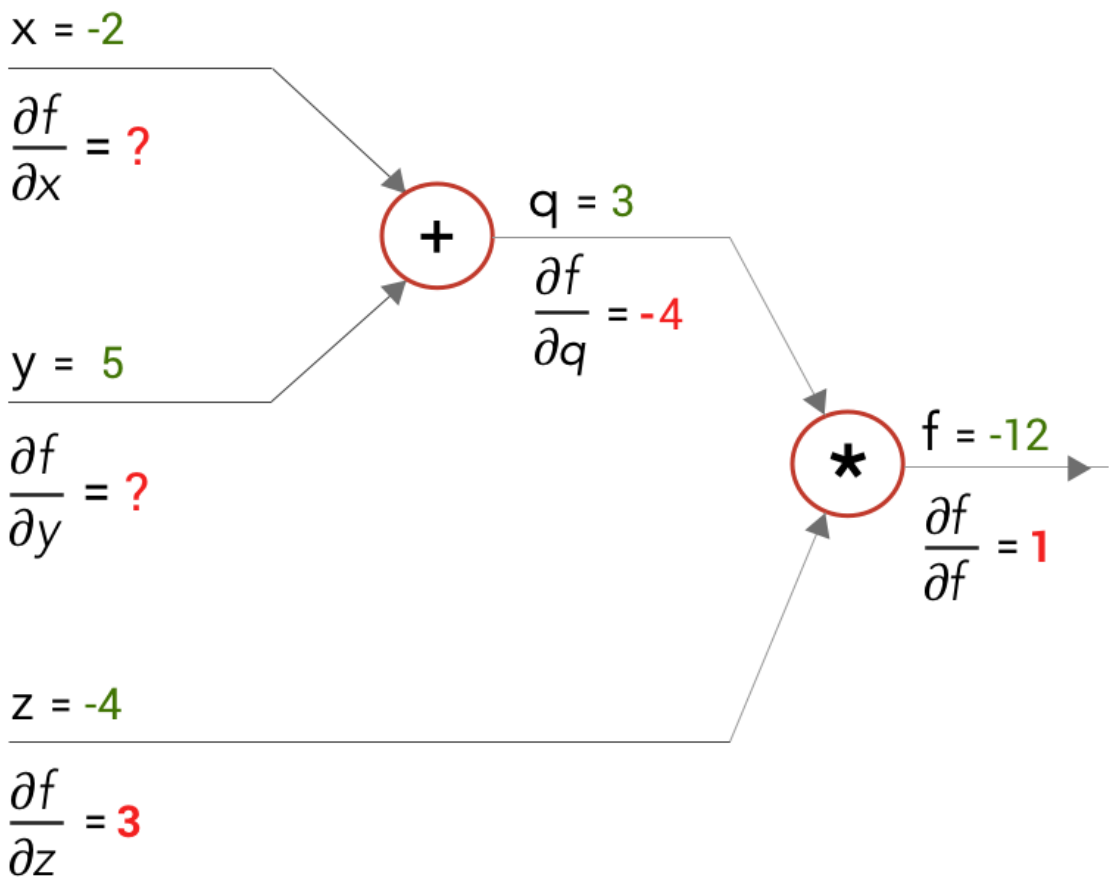
$f = q * z$

$\frac{\partial f}{\partial q} = z \mid z = -4$

$\frac{\partial f}{\partial z} = q \mid q = 3$

$q = x + y$

$\frac{\partial q}{\partial x} = 1$ $\frac{\partial q}{\partial y} = 1$



Example

- Use the chain rule to compute ∂f/∂x and ∂f/∂y:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial x} = -4 * 1 = -4$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial y} = -4 * 1 = -4$$

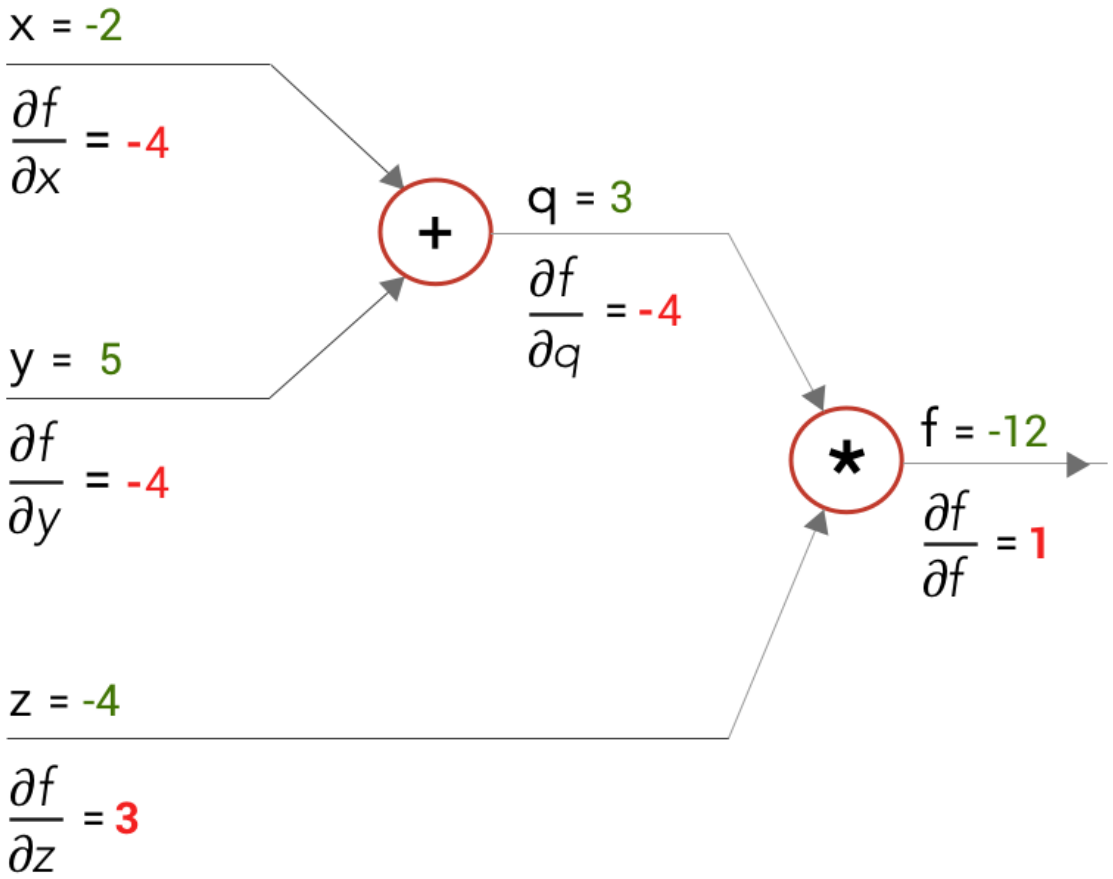
$f = q * z$

$\frac{\partial f}{\partial q} = z \mid z = -4$

$\frac{\partial f}{\partial z} = q \mid q = 3$

$q = x + y$

$\frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$



Another example: Logistic regression

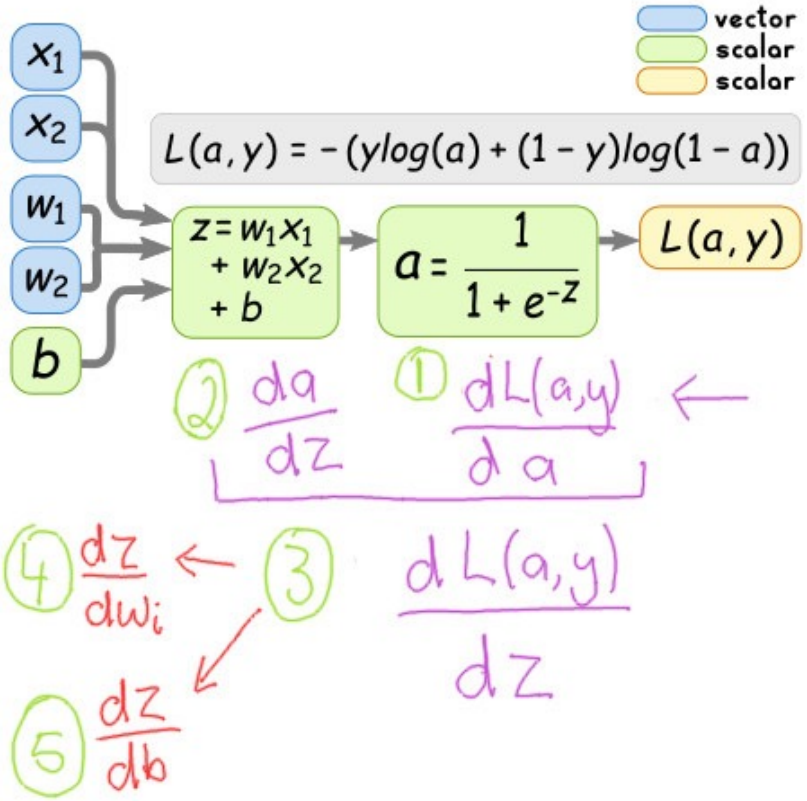
- Here is a detailed walk-through of how to calculate the derivatives in logistic regression.

Desired partial derivatives

$\frac{dL(a,y)}{dw_i}$ } Partial derivatives with respect to the weights

$\frac{dL(a,y)}{db}$ } Partial derivatives with respect to the bias

Strategy for solving partial derivatives



Chain rule (applied twice)

$$\frac{\delta L(a,y)}{\delta w_i} = \frac{\delta L(a,y)}{\delta a} \times \frac{\delta a}{\delta z} \times \frac{\delta z}{\delta w_1}$$

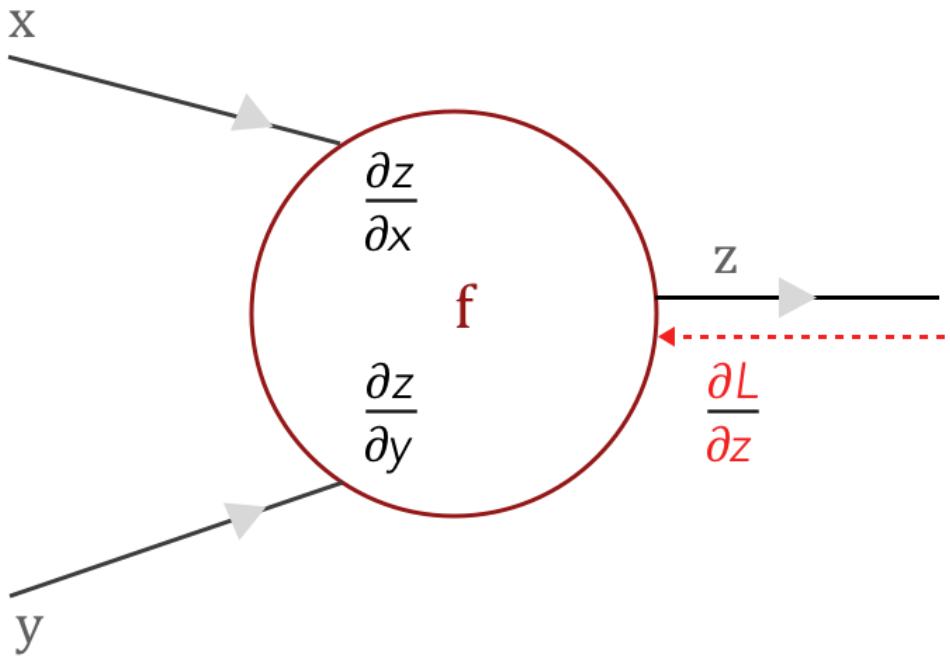
$$\frac{\delta L(a,y)}{\delta b} = \frac{\delta L(a,y)}{\delta a} \times \frac{\delta a}{\delta z} \times \frac{\delta z}{\delta b}$$

$$\frac{\delta L(a,y)}{\delta z}$$

http://ronny.rest/blog/post_2017_08_12

Chain rule in a neuron

- Now that we have worked through a simple computational graph, we can imagine a neural network as a massive computational graph.
- Let us say we have a “neuron” f in that computational graph with inputs x and y which outputs z.
- We can easily compute the local gradients — differentiating z with respect to x and y as ∂z/∂x and ∂z/∂y.
- From the forward pass, we obtain the loss (L).
- When we start to work the loss backwards, we get the gradient of the loss from the previous layer as ∂L/∂z.
- In order for the loss to be propagated to the other gates, we need to find ∂L/∂x and ∂L/∂y.

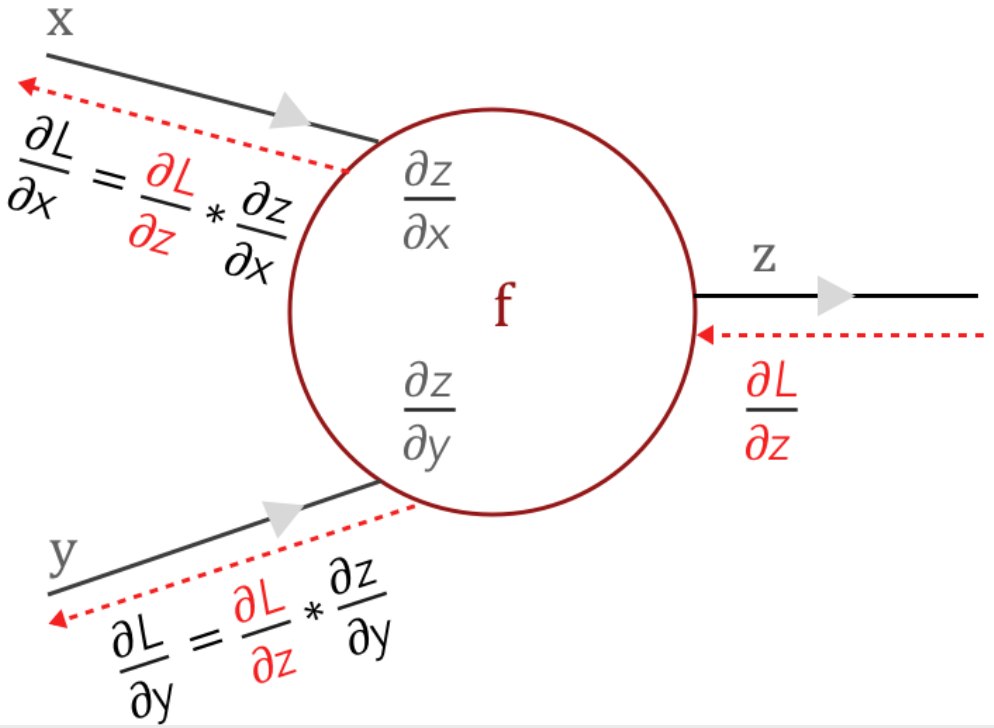


$\frac{\partial z}{\partial x}$ & $\frac{\partial z}{\partial y}$ *are local gradients*

$\frac{\partial L}{\partial z}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Chain rule in a neuron

- The chain rule comes to our help.
- Using the chain rule we can calculate ∂L/∂x and ∂L/∂y, which would feed the other gates in the extended computational graph.
- Defining new autograd functions in PyTorch: https://pytorch.org/tutorials/beginner/examples_autograd/two_layer_net_custom_function.html



$\frac{\partial z}{\partial x}$ & $\frac{\partial z}{\partial y}$ *are local gradients*

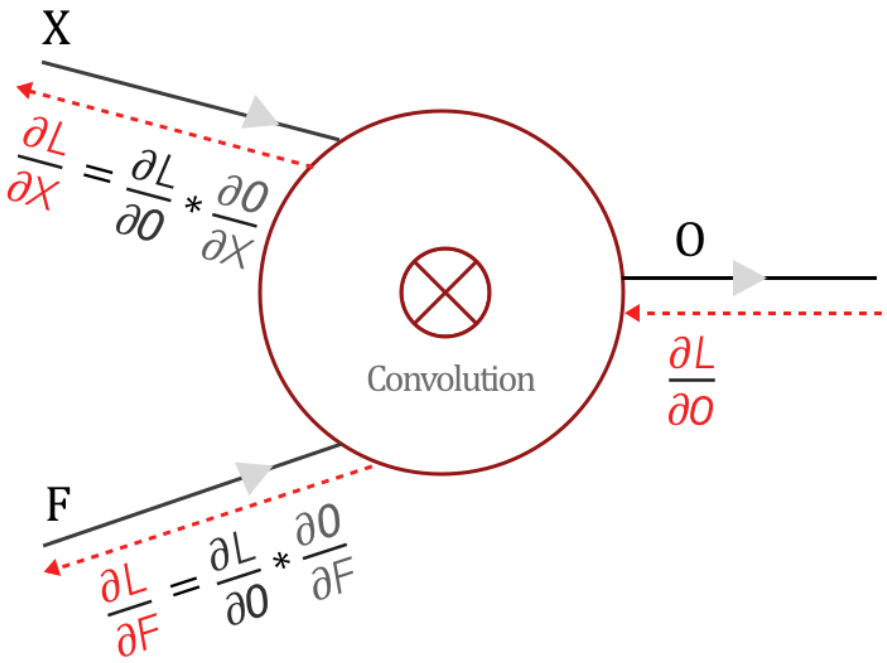
$\frac{\partial L}{\partial z}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

- Now, lets assume the function f is a convolution between Input X and a Filter F.
- Input X is a 3x3 matrix, Filter F is a 2x2 matrix, and Output is a 2x2 matrix.

$$\begin{bmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{bmatrix} = \text{Convolution}\left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

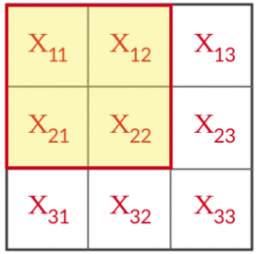
Output O — Input X — Filter F



Backpropagation for convolution

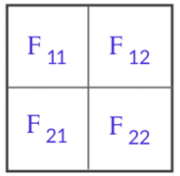
- Forward pass:

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$
$$O_{12} = X_{12}\,F_{11} + X_{13}\,F_{12} + X_{22}\,F_{21} + X_{23}\,F_{22}$$
$$O_{21} = X_{21}\,F_{11} + X_{22}\,F_{12} + X_{31}\,F_{21} + X_{32}\,F_{22}$$
$$O_{22} = X_{22}\,F_{11} + X_{23}\,F_{12} + X_{32}\,F_{21} + X_{33}\,F_{22}$$

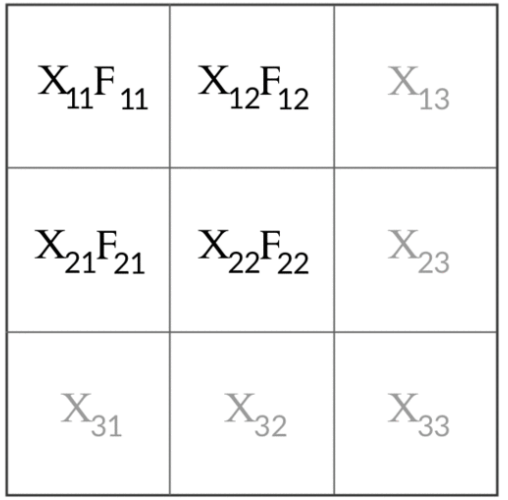


Input X

⊗



Filter F



$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

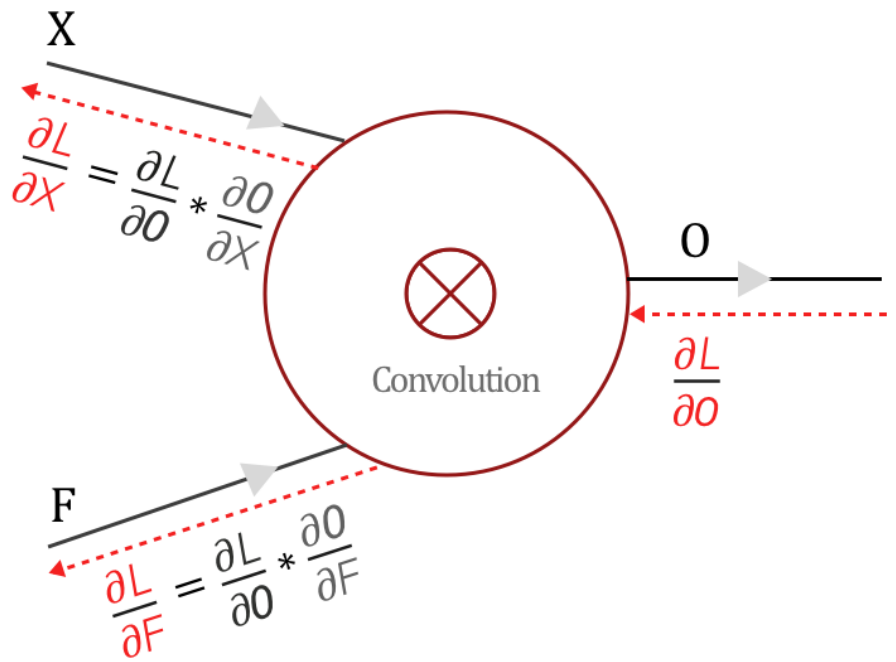
Link to gif animation

Backpropagation for convolution

- Forward pass:

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$
$$O_{12} = X_{12}\,F_{11} + X_{13}\,F_{12} + X_{22}\,F_{21} + X_{23}\,F_{22}$$
$$O_{21} = X_{21}\,F_{11} + X_{22}\,F_{12} + X_{31}\,F_{21} + X_{32}\,F_{22}$$
$$O_{22} = X_{22}\,F_{11} + X_{23}\,F_{12} + X_{32}\,F_{21} + X_{33}\,F_{22}$$

- Backward pass:
  - We get the loss gradient with respect to the Output O from the next layer as ∂L/∂O, during Backward pass.
  - And combining with our previous knowledge using the chain rule and backpropagation, we get what you see in the figure to the right:



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

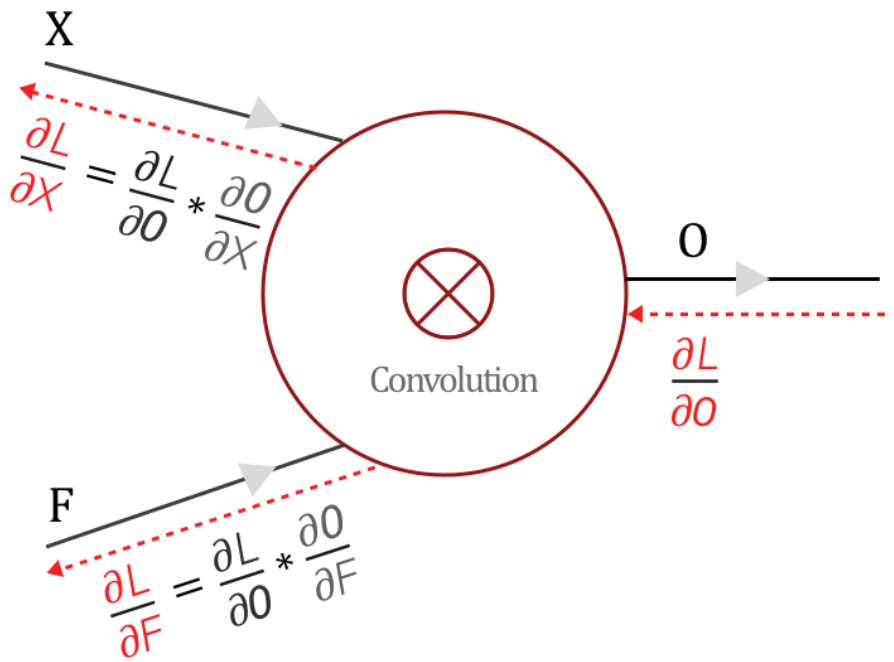
- Finding ∂L/∂F has two steps, just like earlier.
  - Find the local gradient ∂O/∂F
  - Find ∂L/∂F using chain rule
- Example:
  - From forward pass, we know that

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$

  - Then

$$\frac{\partial O_{11}}{\partial F_{11}} = X_{11} \quad \frac{\partial O_{11}}{\partial F_{12}} = X_{12} \quad \frac{\partial O_{11}}{\partial F_{21}} = X_{21} \quad \frac{\partial O_{11}}{\partial F_{22}} = X_{22}$$

- Similarly we can find gradients for O_{12}, O_{21}, and O_{22}.
- This gives us ∂O/∂F (which is a 2D tensor = matrix).



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

- Finding ∂L/∂F has two steps, just like earlier.
  - Find the local gradient ∂O/∂F
  - Find ∂L/∂F using chain rule
- Example:
  - From forward pass, we know that

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$

  - Then

$$\frac{\partial O_{11}}{\partial F_{11}} = X_{11} \quad \frac{\partial O_{11}}{\partial F_{12}} = X_{12} \quad \frac{\partial O_{11}}{\partial F_{21}} = X_{21} \quad \frac{\partial O_{11}}{\partial F_{22}} = X_{22}$$

- Similarly we can find gradients for O_{12}, O_{21}, and O_{22}.
- This gives us ∂O/∂F.

For every element of F

Chain rule

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}}$$

https://www.lesswrong.com/posts/u4DWp72GyLsQn24oR/a-primer-on-matrix-calculus-part-3-the-chain-rule

Backpropagation for convolution

- Finding ∂L/∂F has two steps, just like earlier.
  - Find the local gradient ∂O/∂F
  - Find ∂L/∂F using chain rule
- Example:
  - From forward pass, we know that

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$

  - Then

Substitute

$$\frac{\partial O_{11}}{\partial F_{11}} = X_{11} \quad \frac{\partial O_{11}}{\partial F_{12}} = X_{12} \quad \frac{\partial O_{11}}{\partial F_{21}} = X_{21} \quad \frac{\partial O_{11}}{\partial F_{22}} = X_{22}$$

- Similarly we can find gradients for O_{12}, O_{21}, and O_{22}.
- This gives us ∂O/∂F.

For every element of F

Chain rule

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}}$$

Backpropagation for convolution

For every element of F

Chain rule

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}}$$

Backpropagation for convolution

$$\begin{bmatrix} \frac{\partial L}{\partial F_{11}} & \frac{\partial L}{\partial F_{12}} \\ \frac{\partial L}{\partial F_{21}} & \frac{\partial L}{\partial F_{22}} \end{bmatrix} = \text{Convolution}\left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{bmatrix} \right)$$

Chain rule

For every element of F

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}}$$

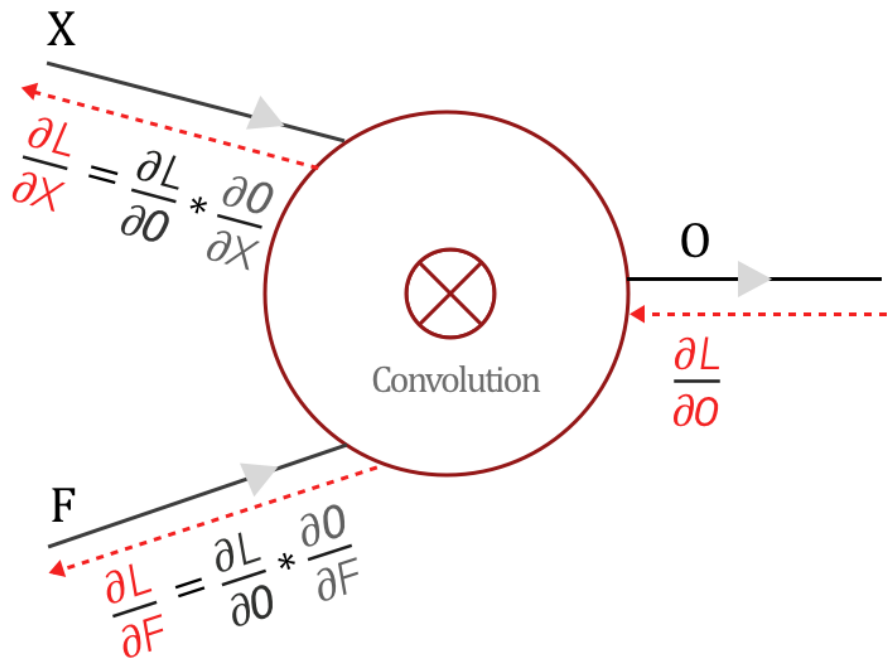
$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}}$$

Backpropagation for convolution

- Finding ∂L/∂F has two steps, just like earlier.
  - Find the local gradient ∂O/∂F
  - Find ∂L/∂F using chain rule
- Result
  - ∂L/∂F is nothing but the convolution between Input X and Loss gradient ∂L/∂O from the layer above:

$$\begin{bmatrix} \frac{\partial L}{\partial F_{11}} & \frac{\partial L}{\partial F_{12}} \\ \frac{\partial L}{\partial F_{21}} & \frac{\partial L}{\partial F_{22}} \end{bmatrix} = \text{Convolution} \left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{bmatrix} \right)$$



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

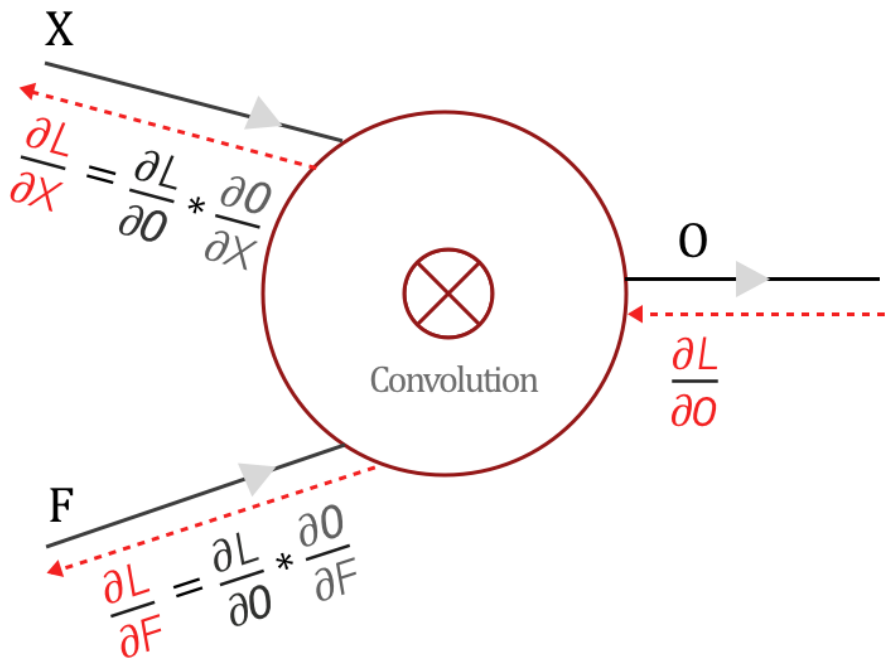
- Finding ∂L/∂X also has two steps:
  - Find the local gradient ∂O/∂X:
  - Find ∂L/∂X using chain rule
- Example – same procedure as before
  - From forward pass, we know that

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$

  - Then

$$\frac{\partial O_{11}}{\partial X_{11}} = F_{11} \quad \frac{\partial O_{11}}{\partial X_{12}} = F_{12} \quad \frac{\partial O_{11}}{\partial X_{21}} = F_{21} \quad \frac{\partial O_{11}}{\partial X_{22}} = F_{22}$$

- Similarly we can find gradients for O_{12}, O_{21}, and O_{22}.
- This gives us ∂O/∂X.



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

- Finding ∂L/∂X also has two steps:
  - Find the local gradient ∂O/∂X:
  - Find ∂L/∂X using chain rule
- Example – same procedure as before
  - From forward pass, we know that

$$O_{11} = X_{11}\,F_{11} + X_{12}\,F_{12} + X_{21}\,F_{21} + X_{22}\,F_{22}$$

  - Then

$$\frac{\partial O_{11}}{\partial X_{11}} = F_{11} \quad \frac{\partial O_{11}}{\partial X_{12}} = F_{12} \quad \frac{\partial O_{11}}{\partial X_{21}} = F_{21} \quad \frac{\partial O_{11}}{\partial X_{22}} = F_{22}$$

- Similarly we can find gradients for O_{12}, O_{21}, and O_{22}.
- This gives us ∂O/∂X.

Chain rule

For every element of X_i

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial X_i}$$

Backpropagation for convolution

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} * F_{11}$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} * F_{12} + \frac{\partial L}{\partial O_{12}} * F_{11}$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{12}} * F_{12}$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{11}$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} * F_{22} + \frac{\partial L}{\partial O_{12}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{12} + \frac{\partial L}{\partial O_{22}} * F_{11}$$

$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{12}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{12}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{21}} * F_{21}$$

$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{21}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{21}$$

$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{22}} * F_{22}$$

Expanding and substituting

Chain rule

For every element of X_i

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial X_i}$$

Backpropagation for convolution

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} * F_{11}$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} * F_{12} + \frac{\partial L}{\partial O_{12}} * F_{11}$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{12}} * F_{12}$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{11}$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} * F_{22} + \frac{\partial L}{\partial O_{12}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{12} + \frac{\partial L}{\partial O_{22}} * F_{11}$$

$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{12}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{12}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{21}} * F_{21}$$

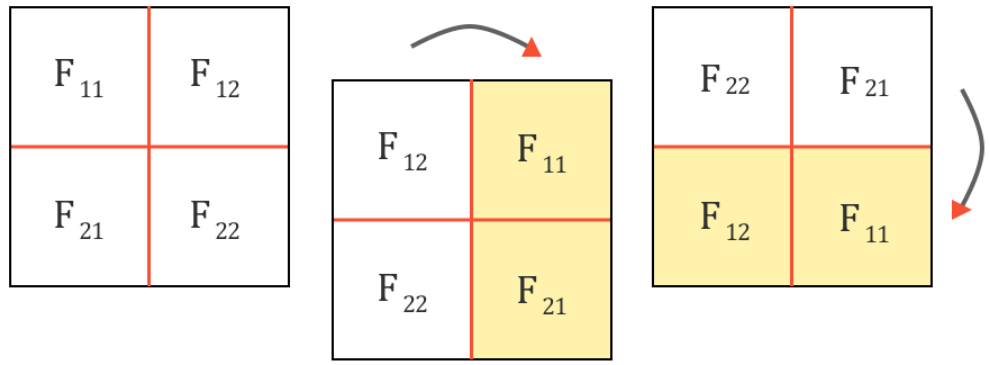
$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{21}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{21}$$

$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{22}} * F_{22}$$

Ok. Now we have the values of ∂L/∂X.

Believe it or not, even this can be represented as a convolution operation.

∂L/∂X can be represented as ‘full’ convolution between a 180-degree rotated Filter F and loss gradient ∂L/∂O



Backpropagation for convolution

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} * F_{11}$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} * F_{12} + \frac{\partial L}{\partial O_{12}} * F_{11}$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{12}} * F_{12}$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{11}$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} * F_{22} + \frac{\partial L}{\partial O_{12}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{12} + \frac{\partial L}{\partial O_{22}} * F_{11}$$

$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{12}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{12}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{21}} * F_{21}$$

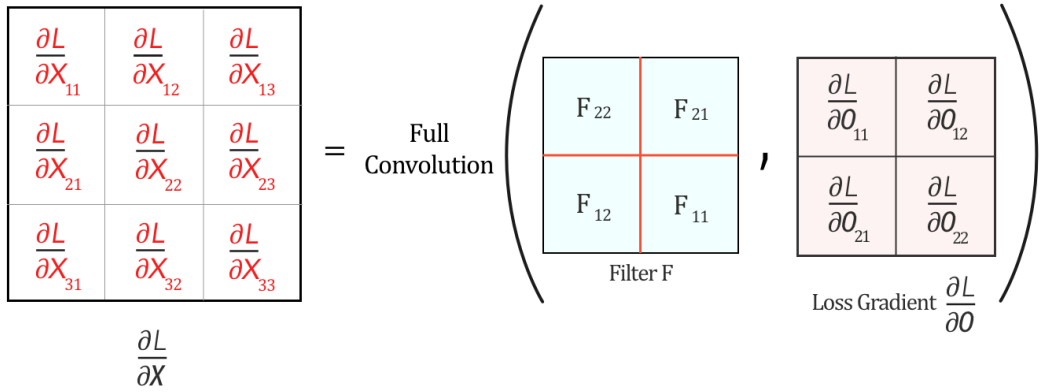
$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{21}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{21}$$

$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{22}} * F_{22}$$

Ok. Now we have the values of ∂L/∂X.

Believe it or not, even this can be represented as a convolution operation.

∂L/∂X can be represented as ‘full’ convolution between a 180-degree rotated Filter F and loss gradient ∂L/∂O



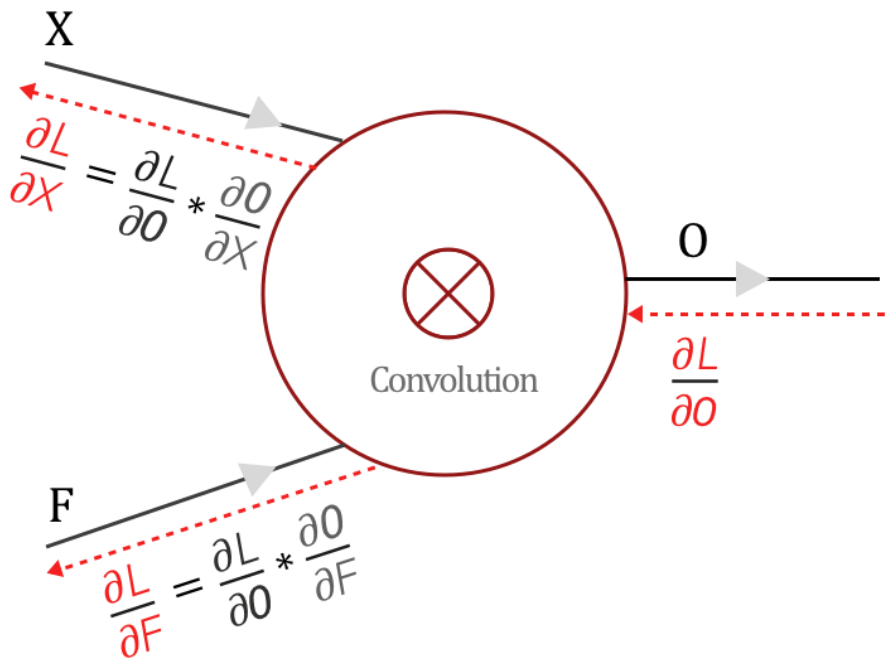
Backpropagation for convolution

- Finding ∂L/∂X also has two steps:
  - Find the local gradient ∂O/∂X:
  - Find ∂L/∂X using chain rule
- Result
  - ∂L/∂X can be represented as 'full' convolution between a 180-degree rotated Filter F and loss gradient ∂L/∂O:

$$\frac{\partial L}{\partial X} = \begin{bmatrix} \frac{\partial L}{\partial X_{11}} & \frac{\partial L}{\partial X_{12}} & \frac{\partial L}{\partial X_{13}} \\ \frac{\partial L}{\partial X_{21}} & \frac{\partial L}{\partial X_{22}} & \frac{\partial L}{\partial X_{23}} \\ \frac{\partial L}{\partial X_{31}} & \frac{\partial L}{\partial X_{32}} & \frac{\partial L}{\partial X_{33}} \end{bmatrix} = \text{Full Convolution} \left(\begin{bmatrix} F_{22} & F_{21} \\ F_{12} & F_{11} \end{bmatrix}, \begin{bmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{bmatrix} \right)$$

Filter F

Loss Gradient $\frac{\partial L}{\partial O}$



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Backpropagation for convolution

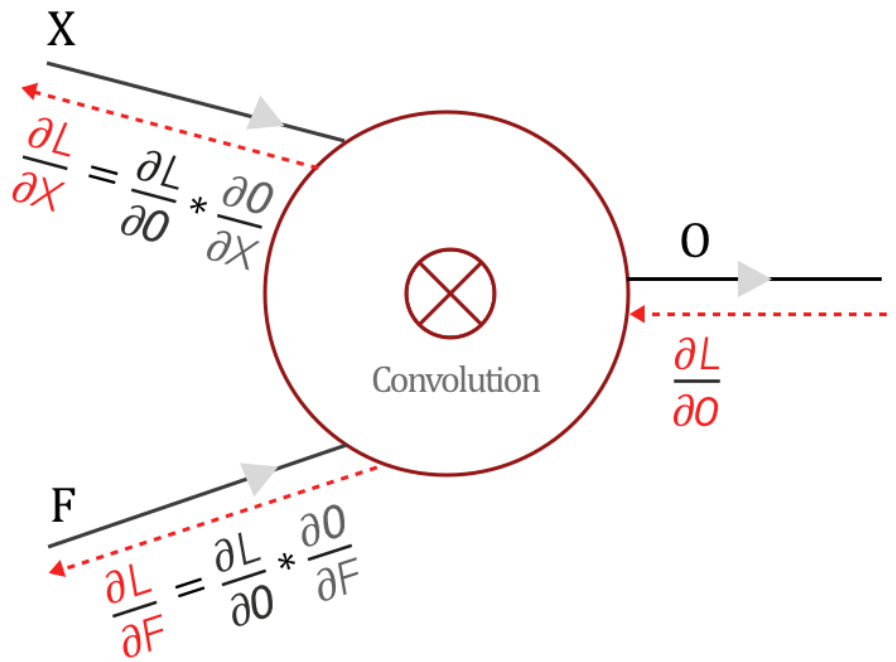
- Summing it up:

Backpropagation in a Convolutional Layer of a CNN

Finding the gradients:

$$\frac{\partial L}{\partial F} = \text{Convolution}\left(\text{Input } X\text{ , Loss gradient } \frac{\partial L}{\partial O}\right)$$

$$\frac{\partial L}{\partial X} = \text{Full Convolution}\left(180^{\circ}\text{rotated Filter } F\text{ , Loss Gradient } \frac{\partial L}{\partial O}\right)$$



$\frac{\partial O}{\partial X}$ & $\frac{\partial O}{\partial F}$ *are local gradients*

$\frac{\partial L}{\partial O}$ *is the loss from the previous layer which has to be backpropagated to other layers*

Further reading + online videos/demos

- Derivation of the backpropagation algorithm
  - http://neuralnetworksanddeeplearning.com/chap2.html
- Cross-entropy, regularization and other practical considerations:
  - http://neuralnetworksanddeeplearning.com/chap3.html
- Machine Learning, Andrew Ng (Stanford), full course:
  - https://www.youtube.com/playlist?list=PLLssT5z_DsK-h9vYZkQkYNWcItqhlRJLN
- Deep learning frameworks like PyTorch and Keras perform automatic differentiation/backprop. We will talk about it later in the course. Here are a few useful links if you are interested:
  - https://blog.paperspace.com/pytorch-101-understanding-graphs-and-automatic-differentiation/
  - https://towardsdatascience.com/pytorch-autograd-understanding-the-heart-of-pytorchs-magic-2686cd94ec95
  - https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
  - https://pytorch.org/tutorials/beginner/examples_autograd/two_layer_net_custom_function.html

Further reading + online videos/demos

- Computational graphs and backpropagation in CNNs:
  - https://becominghuman.ai/back-propagation-in-convolutional-neural-networks-intuition-and-code-714ef1c38199
  - https://medium.com/@pavisj/convolutions-and-backpropagations-46026a8f5d2c
  - http://cs231n.github.io/optimization-2/
  - http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture04.pdf